CEE6501 — Lecture 11.3

Coding Geometric Nonlinearity with the DSM

Learning Objectives

By the end of this lecture, you will be able to:

- Implement Newton-Raphson in Python
- Solve a geometrically nonlinear 2D truss problem

Agenda

- Environment Setup
- Problem Definition and Setup
- Iteration 0
- Iteration 1
- Iteration 2
- Iteration 3 (Empty)

Environment Setup

In [1]:

```
# -------------------------
# Import Libraries and Helper Functions
# -------------------------
# This slide imports the required Python libraries and adds the
# course helper-code directory to the Python path.

import sys
import os
import numpy as np
import pandas as pd
```

In [2]:
```
# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)

# Go up two levels, then into Code/L11
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L11")
)

sys.path.append(helpers_path)
```

In [3]:
```
import numpy as np

from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    assemble_global_displacements,
    partition_system,
)
from helper_funcs_output import (
    print_matrix_scaled,
    print_vector_scaled
)
from helpers_nonlinear_truss import (
    geometry_from_nodes,
    k_global_2d_truss_elastic,
    k_global_2d_truss_geometric
)
```

Problem Definition and Setup

In [4]:
```
# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 70e6          # kPa, kN/m^2
A = 645.2e-6       # m^2

nb_1 = [0, 0]
ne_1 = [4, 3]

nb_2 = [8, 0]
ne_2 = [4, 3]

nb_3 = [0, 0]
ne_3 = [8, 0]

e = 0.01 # Convergence criterion, 1% change in displacements
```

In [5]:
```
# -------------------------
# Problem data (hard-coded)
# -------------------------
n_nodes = 3
ndof_per_node = 2

# Global system size (still using 1-based maps here)
ndof = n_nodes * ndof_per_node

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [6]:

```
# -------------------------
# User-Specified Boundary Conditions
# -------------------------

# Applied Load (0-indexed)
F_global[4-1] = -2000 #kN*m

# Restrained DOFs
dof_restrained_1based = np.array([1, 2, 6], dtype=int)

print(F_global)
```

```
[    0.     0.     0. -2000.     0.     0.]
```

In [7]:
```
# -------------------------
# User-Specified DOF Mapping
# -------------------------

# Mapping
m1 = np.array([1,2,3,4])
m2 = np.array([5,6,3,4])
m3 = np.array([1,2,5,6])

map_list = [m1, m2, m3]   # 1-based DOF maps
```

Iteration 0

In [8]:

```
# ==========================================================
# NEWTON-RAPHSON ITERATION 0
# ==========================================================
# Lecture alignment:
# Step 1. initialize with tangent stiffness matrix at 0 deformation
# Step 2. solve for u_f^(1) as a linearized first estimate
# ==========================================================
```

Step 1

In [9]:
```
# ----------------------------------------------------------
# Step 1 — Initialize with tangent stiffness at 0 deformation
# ----------------------------------------------------------
# At the start of the analysis, the structure is undeformed.
# So for each member:
#   - current geometry = initial geometry
#   - axial deformation v = 0
#   - axial force q = 0
#   - geometric stiffness contribution = 0
#
# The initial tangent stiffness is therefore formed from the
# undeformed member geometry.
```

In [10]:
```
# -------------------------
# Member 1
# -------------------------
L1, cx1, cy1, T1 = geometry_from_nodes(nb_1, ne_1)

Lbar_1 = np.copy(L1)          # undeformed configuration
v1 = Lbar_1 - L1
q1 = E * A * v1 / L1

k1_e = k_global_2d_truss_elastic(E, A, L1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5781.0   4335.7  -5781.0  -4335.7
02 |   4335.7   3251.8  -4335.7  -3251.8
03 |  -5781.0  -4335.7   5781.0   4335.7
04 |  -4335.7  -3251.8   4335.7   3251.8
K = 1e+00 ×
01 |      0.0     -0.0     -0.0      0.0
02 |     -0.0      0.0      0.0     -0.0
03 |     -0.0      0.0      0.0     -0.0
04 |      0.0     -0.0     -0.0      0.0
```

In [11]:
```
# -------------------------
# Member 2
# -------------------------
L2, cx2, cy2, T2 = geometry_from_nodes(nb_2, ne_2)

Lbar_2 = np.copy(L2)          # undeformed configuration
v2 = Lbar_2 - L2
q2 = E * A * v2 / L2

k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5781.0  -4335.7  -5781.0   4335.7
02 |  -4335.7   3251.8   4335.7  -3251.8
03 |  -5781.0   4335.7   5781.0  -4335.7
04 |   4335.7  -3251.8  -4335.7   3251.8
K = 1e+00 ×
01 |      0.0      0.0     -0.0     -0.0
02 |      0.0      0.0     -0.0     -0.0
03 |     -0.0     -0.0      0.0      0.0
04 |     -0.0     -0.0      0.0      0.0
```

In [12]:
```
# -------------------------
# Member 3
# -------------------------
L3, cx3, cy3, T3 = geometry_from_nodes(nb_3, ne_3)

Lbar_3 = np.copy(L3)          # undeformed configuration
v3 = Lbar_3 - L3
q3 = E * A * v3 / L3

k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5     -0.0
02 |      0.0      0.0     -0.0     -0.0
03 |  -5645.5     -0.0   5645.5      0.0
04 |     -0.0     -0.0      0.0      0.0
K = 1e+00 ×
01 |      0.0     -0.0     -0.0      0.0
02 |     -0.0      0.0      0.0     -0.0
03 |     -0.0      0.0      0.0     -0.0
04 |      0.0     -0.0     -0.0      0.0
```

In [13]:

```
# -------------------------
# Assemble initial tangent stiffness
# -------------------------
k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]

# Placeholder lists for generalized assembly code
T_list = [np.eye(4), np.eye(4), np.eye(4)]
Qf_list = [np.zeros(4), np.zeros(4), np.zeros(4)]

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
)
```

In [14]:

```
# -------------------------
# Display Global Stiffness Matrix
# -------------------------
# Print the assembled global stiffness matrix in a readable format.
# The matrix is scaled for clearer presentation in the slides.

print_matrix_scaled(K_global, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  11426.5   4335.7  -5781.0  -4335.7  -5645.5      0.0
02 |   4335.7   3251.8  -4335.7  -3251.8      0.0      0.0
03 |  -5781.0  -4335.7  11562.0      0.0  -5781.0   4335.7
04 |  -4335.7  -3251.8      0.0   6503.6   4335.7  -3251.8
05 |  -5645.5      0.0  -5781.0   4335.7  11426.5  -4335.7
06 |      0.0      0.0   4335.7  -3251.8  -4335.7   3251.8
```

In [15]:

```
# -------------------------
# Partition to free / restrained DOFs
# -------------------------
(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [16]:

```
# -------------------------
# Print stiffness matrix for free DOFs
# -------------------------

print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  11562.0      0.0  -5781.0
02 |      0.0   6503.6   4335.7
03 |  -5781.0   4335.7  11426.5
```

Step 2

In [17]:

```
# ---------------------------------------------------------
# Step 2 — Solve for u_f^(1), the first approximation
# ---------------------------------------------------------
# This is the linearized first estimate obtained using the
# tangent stiffness in the undeformed configuration:
#
#     u_f^(1) = K_ff,t^{-1(0)} f_f
#
```

In [18]:

```
# -------------------------
# Solve Partitioned System
# -------------------------

rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

u_f_iteration1 = np.copy(u_f)
print_vector_scaled(u_f_iteration1, name="u_f^(1)", scale=1, decimals=4, col_w
```

```
u_f^(1) = 1e+00 × [0.1181 -0.4650 0.2362]
```

In [19]:
```
u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

print_vector_scaled(u_global, name="u^(1)", scale=1, decimals=4, col_width=6)
```

```
u^(1) = 1e+00 × [0.0000 0.0000 0.1181 -0.4650 0.2362 0.0000]
```

Iteration 1

In [20]:

```
# ============================================================
# NEWTON-RAPHSON ITERATION 1
# ============================================================
# Lecture alignment:
# Step 3. compute internal force f_f,int^(1)
# Step 4. compute residual r^(1)
# Step 5. form the new tangent stiffness K_ff,t^(1)
# Step 6. solve for correction du_f^(1)
# Step 7. update to obtain u_f^(2)
# Step 8. check convergence
# Step 9. if not converged, repeat
# ============================================================
```

Step 3

In [21]:
```
# ----------------------------------------------------------
# Step 3 — Compute internal resisting force at u^(1)
# ----------------------------------------------------------
# Evaluate each member in the current deformed configuration,
# then assemble the corresponding internal force vector.
```

In [22]:

```
# -------------------------
# Member 1
# -------------------------
u1 = u_global[m1 - 1]

nb_1_new = np.array(nb_1) + np.array([u1[0], u1[1]])
ne_1_new = np.array(ne_1) + np.array([u1[2], u1[3]])

Lbar_1, cx1, cy1, T1 = geometry_from_nodes(nb_1_new, ne_1_new)

v1 = Lbar_1 - L1
q1 = E * A * v1 / L1
f1 = T1.T * q1

print_vector_scaled(f1, name="f_f,int,1", scale=1, decimals=1, col_width=6)
```

```
f_f,int,1 = 1e+00 × [1263.0  777.5 -1263.0 -777.5]
```

In [23]:

```
# -------------------------
# Member 2
# -------------------------
u2 = u_global[m2 - 1]

nb_2_new = np.array(nb_2) + np.array([u2[0], u2[1]])
ne_2_new = np.array(ne_2) + np.array([u2[2], u2[3]])

Lbar_2, cx2, cy2, T2 = geometry_from_nodes(nb_2_new, ne_2_new)

v2 = Lbar_2 - L2
q2 = E * A * v2 / L2
f2 = T2.T * q2

print_vector_scaled(f2, name="f_f,int,2", scale=1, decimals=1, col_width=6)
```

```
f_f,int,2 = 1e+00 × [-1263.0  777.5 1263.0 -777.5]
```

In [24]:

```
# -------------------------
# Member 3
# -------------------------
u3 = u_global[m3 - 1]

nb_3_new = np.array(nb_3) + np.array([u3[0], u3[1]])
ne_3_new = np.array(ne_3) + np.array([u3[2], u3[3]])

Lbar_3, cx3, cy3, T3 = geometry_from_nodes(nb_3_new, ne_3_new)

v3 = Lbar_3 - L3
q3 = E * A * v3 / L3
f3 = T3.T * q3

print_vector_scaled(f3, name="f_f,int,3", scale=1, decimals=1, col_width=6)
```

```
f_f,int,3 = 1e+00 × [-1333.3   -0.0 1333.3    0.0]
```

In [25]:
```
# Assemble internal force vector
f_internal = np.zeros(ndof, dtype=float)
f_internal[np.array(m1) - 1] += np.asarray(f1).reshape(-1)
f_internal[np.array(m2) - 1] += np.asarray(f2).reshape(-1)
f_internal[np.array(m3) - 1] += np.asarray(f3).reshape(-1)

print_vector_scaled(f_internal, name="f_int", scale=1, decimals=1, col_width=6
```

```
f_int = 1e+00 × [ -70.3  777.5    0.0 -1555.0   70.3  777.5]
```

Step 4

In [26]:
```
# ---------------------------------------------------------
# Step 4 — Compute residual r^(1)
# ---------------------------------------------------------
# Residual = applied load - internal resisting force
# evaluated at the free DOFs.
r_iteration1 = f_f - f_internal[free_dofs]
print_vector_scaled(r_iteration1, name="r^(1)", scale=1, decimals=3, col_width
```

```
r^(1) = 1e+00 × [ 0.000 -445.021 -70.322]
```

Step 5

In [27]:
```
# ---------------------------------------------------------
# Step 5 — Form the new tangent stiffness K_ff,t^(1)
# ---------------------------------------------------------
```

In [28]:

```
# -------------------------
# Member 1
# -------------------------
k1_e = k_global_2d_truss_elastic(E, A, L1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e + k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6466.2   4169.3  -6466.2  -4169.3
02 |   4169.3   2259.9  -4169.3  -2259.9
03 |  -6466.2  -4169.3   6466.2   4169.3
04 |  -4169.3  -2259.9   4169.3   2259.9
```

In [29]:

```
# -------------------------
# Member 2
# -------------------------
k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e + k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6466.2  -4169.3  -6466.2   4169.3
02 |  -4169.3   2259.9   4169.3  -2259.9
03 |  -6466.2   4169.3   6466.2  -4169.3
04 |   4169.3  -2259.9  -4169.3   2259.9
```

In [30]:

```
# -------------------------
# Member 3
# -------------------------
k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e + k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5      0.0
02 |      0.0    161.9      0.0   -161.9
03 |  -5645.5      0.0   5645.5      0.0
04 |      0.0   -161.9      0.0    161.9
```

In [31]:

```
# Reassemble the global tangent stiffness using the current
# member geometry and current axial force state.

k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
)
```

In [32]:

```
# -------------------------
# Partition to free / restrained DOFs
# -------------------------

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [33]:

```
# -------------------------
# Print stiffness matrix for free DOFs
# -------------------------

print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  12932.5      0.0  -6466.2
02 |      0.0   4519.7   4169.3
03 |  -6466.2   4169.3  12111.7
```

Step 6

In [34]:

```
# ---------------------------------------------------------
# Step 6 — Solve for correction du_f^(1)
# ---------------------------------------------------------
# Solve:
#     K_ff,t^(1) du_f^(1) = r^(1)
rhs = r_iteration1 - f_fef_f - K_fr @ u_r
du_f_iteration1 = np.linalg.solve(K_ff, rhs)

print_vector_scaled(du_f_iteration1, name="du_f^(1)", scale=1, decimals=6, col
```

```
du_f^(1) = 1e+00 × [0.033800 -0.160820 0.067599]
```

Step 7

In [35]:
```
# ---------------------------------------------------------
# Step 7 — Update to obtain u_f^(2)
# ---------------------------------------------------------
u_f_iteration2 = u_f_iteration1 + du_f_iteration1

u_global = assemble_global_displacements(
    u_f_iteration2,
    u_r,
    free_dofs,
    restrained_dofs
)

print_vector_scaled(u_global, name="u^(2)", scale=1, decimals=5, col_width=6)
print_vector_scaled(u_f_iteration2, name="u_f^(2)", scale=1, decimals=5, col_w
```

```
u^(2) = 1e+00 × [0.00000 0.00000 0.15189 -0.62579 0.30378 0.00000]
u_f^(2) = 1e+00 × [0.15189 -0.62579 0.30378]
```

Step 8

In [36]:

```
# ---------------------------------------------------------
# Step 8 — Check convergence
# ---------------------------------------------------------
# Displacement-based convergence check from lecture:
#
#   sqrt( sum(du_f^(i)^2) / sum(u_f^(i)^2) ) <= e
#
value = np.sqrt(np.sum(du_f_iteration1**2) / np.sum(u_f_iteration1**2))
print("Convergence value:", value)

converged = value <= e
print("Converged:", converged)
```

```
Convergence value: 0.3323144606446154
Converged: False
```

Step 9

In [37]:
```
# ---------------------------------------------------------
# Step 9 — If not converged, repeat Steps 3–8
# ---------------------------------------------------------
# Since this is a hardcoded lecture example, we now move
# manually to Iteration 2.
```

Iteration 2

In [38]:

```
# ============================================================
# NEWTON-RAPHSON ITERATION 2
# ============================================================
# Lecture alignment:
# Step 3. compute internal force f_f,int^(2)
# Step 4. compute residual r^(2)
# Step 5. form the new tangent stiffness K_ff,t^(2)
# Step 6. solve for correction du_f^(2)
# Step 7. update to obtain u_f^(2)
# Step 8. check convergence
# Step 9. if not converged, repeat
# ============================================================
```

Step 3

In [39]:
```
# ----------------------------------------------------------
# Step 3 — Compute internal resisting force at u^(2)
# ----------------------------------------------------------
# Evaluate each member in the current deformed configuration,
# then assemble the corresponding internal force vector.
```

In [40]:

```
# -------------------------
# Member 1
# -------------------------
u1 = u_global[m1 - 1]

nb_1_new = np.array(nb_1) + np.array([u1[0], u1[1]])
ne_1_new = np.array(ne_1) + np.array([u1[2], u1[3]])

Lbar_1, cx1, cy1, T1 = geometry_from_nodes(nb_1_new, ne_1_new)

v1 = Lbar_1 - L1
q1 = E * A * v1 / L1
f1 = T1.T * q1

print_vector_scaled(f1, name="f_f,int,1", scale=1, decimals=1, col_width=6)
```

```
f_f,int,1 = 1e+00 × [1703.2  974.0 -1703.2 -974.0]
```

In [41]:

```
# -------------------------
# Member 2
# -------------------------
u2 = u_global[m2 - 1]

nb_2_new = np.array(nb_2) + np.array([u2[0], u2[1]])
ne_2_new = np.array(ne_2) + np.array([u2[2], u2[3]])

Lbar_2, cx2, cy2, T2 = geometry_from_nodes(nb_2_new, ne_2_new)

v2 = Lbar_2 - L2
q2 = E * A * v2 / L2
f2 = T2.T * q2

print_vector_scaled(f2, name="f_f,int,2", scale=1, decimals=1, col_width=6)
```

```
f_f,int,2 = 1e+00 × [-1703.2  974.0 1703.2 -974.0]
```

In [42]:
```
# -------------------------
# Member 3
# -------------------------
u3 = u_global[m3 - 1]

nb_3_new = np.array(nb_3) + np.array([u3[0], u3[1]])
ne_3_new = np.array(ne_3) + np.array([u3[2], u3[3]])

Lbar_3, cx3, cy3, T3 = geometry_from_nodes(nb_3_new, ne_3_new)

v3 = Lbar_3 - L3
q3 = E * A * v3 / L3
f3 = T3.T * q3

print_vector_scaled(f3, name="f_f,int,3", scale=1, decimals=1, col_width=6)
```

```
f_f,int,3 = 1e+00 × [-1715.0   -0.0 1715.0    0.0]
```

In [43]:
```
# Assemble internal force vector
f_internal = np.zeros(ndof, dtype=float)
f_internal[np.array(m1) - 1] += np.asarray(f1).reshape(-1)
f_internal[np.array(m2) - 1] += np.asarray(f2).reshape(-1)
f_internal[np.array(m3) - 1] += np.asarray(f3).reshape(-1)

print_vector_scaled(f_internal, name="f_int", scale=1, decimals=1, col_width=6
```

```
f_int = 1e+00 × [ -11.7  974.0    0.0 -1948.0   11.7  974.0]
```

Step 4

In [44]:
```
# ---------------------------------------------------------
# Step 4 — Compute residual r^(1)
# ---------------------------------------------------------
# Residual = applied load - internal resisting force
# evaluated at the free DOFs.
r_iteration2 = f_f - f_internal[free_dofs]
print_vector_scaled(r_iteration2, name="r^(1)", scale=1, decimals=3, col_width
```

```
r^(1) = 1e+00 × [ 0.000 -52.041 -11.722]
```

Step 5

In [45]:
```
# ---------------------------------------------------------
# Step 5 — Form the new tangent stiffness K_ff,t^(1)
# ---------------------------------------------------------
```

In [46]:

```
# -------------------------
# Member 1
# -------------------------
k1_e = k_global_2d_truss_elastic(E, A, L1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e + k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6705.8   4069.2  -6705.8  -4069.2
02 |   4069.2   1916.7  -4069.2  -1916.7
03 |  -6705.8  -4069.2   6705.8   4069.2
04 |  -4069.2  -1916.7   4069.2   1916.7
```

In [47]:

```
# -------------------------
# Member 2
# -------------------------
k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e + k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6705.8  -4069.2  -6705.8   4069.2
02 |  -4069.2   1916.7   4069.2  -1916.7
03 |  -6705.8   4069.2   6705.8  -4069.2
04 |   4069.2  -1916.7  -4069.2   1916.7
```

In [48]:

```
# -------------------------
# Member 3
# -------------------------
k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e + k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5      0.0
02 |      0.0    206.5      0.0   -206.5
03 |  -5645.5      0.0   5645.5      0.0
04 |      0.0   -206.5      0.0    206.5
```

In [49]:

```
# Reassemble the global tangent stiffness using the current
# member geometry and current axial force state.

k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
)
```

In [50]:

```
# -------------------------
# Partition to free / restrained DOFs
# -------------------------

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [51]:

```
# -------------------------
# Print stiffness matrix for free DOFs
# -------------------------

print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  13411.7      0.0  -6705.8
02 |      0.0   3833.4   4069.2
03 |  -6705.8   4069.2  12351.3
```

Step 6

In [52]:

```
# ---------------------------------------------------------
# Step 6 — Solve for correction du_f^(2)
# ---------------------------------------------------------
# Solve:
#     K_ff,t^(2) du_f^(2) = r^(2)
rhs = r_iteration2 - f_fef_f - K_fr @ u_r
du_f_iteration2 = np.linalg.solve(K_ff, rhs)

print_vector_scaled(du_f_iteration2, name="du_f^(2)", scale=1, decimals=6, col
```

```
du_f^(2) = 1e+00 × [0.004651 -0.023449 0.009302]
```

Step 7

In [53]:
```
# ---------------------------------------------------------
# Step 7 — Update to obtain u_f^(3)
# ---------------------------------------------------------
u_f_iteration3 = u_f_iteration2 + du_f_iteration2

u_global = assemble_global_displacements(
    u_f_iteration3,
    u_r,
    free_dofs,
    restrained_dofs
)

print_vector_scaled(u_global, name="u^(3)", scale=1, decimals=5, col_width=6)
print_vector_scaled(u_f_iteration3, name="u_f^(3)", scale=1, decimals=5, col_w
```

```
u^(3) = 1e+00 × [0.00000 0.00000 0.15654 -0.64924 0.31308 0.00000]
u_f^(3) = 1e+00 × [0.15654 -0.64924 0.31308]
```

Step 8

In [54]:

```
# ---------------------------------------------------------
# Step 8 — Check convergence
# ---------------------------------------------------------
# Displacement-based convergence check from lecture:
#
#   sqrt( sum(du_f^(i)^2) / sum(u_f^(i)^2) ) <= e
#
value = np.sqrt(np.sum(du_f_iteration2**2) / np.sum(u_f_iteration2**2))
print("Convergence value:", value)

converged = value <= e
print("Converged:", converged)
```

```
Convergence value: 0.036027098504767145
Converged: False
```

Step 9

In [55]:
```
# ---------------------------------------------------------
# Step 9 — If not converged, repeat Steps 3–8
# ---------------------------------------------------------
# Since this is a hardcoded lecture example, we now move
# manually to Iteration 2.
```

Iteration 3

In [56]:

```
# ============================================================
# NEWTON-RAPHSON ITERATION 2
# ============================================================
# Lecture alignment:
# Step 3. compute internal force f_f,int^(2)
# Step 4. compute residual r^(2)
# Step 5. form the new tangent stiffness K_ff,t^(2)
# Step 6. solve for correction du_f^(2)
# Step 7. update to obtain u_f^(2)
# Step 8. check convergence
# Step 9. if not converged, repeat
# ============================================================
```

Step 3

In [57]:
```
# ----------------------------------------------------------
# Step 3 — Compute internal resisting force at u^(2)
# ----------------------------------------------------------
# Evaluate each member in the current deformed configuration,
# then assemble the corresponding internal force vector.
```

Complete in class if time...